

PROJET DATA LAKE

INGE2-BDML

**AI DOCUMENT INTELLIGENT
DATA LAKE POUR RAG FINANCIER**

2025-2026

Présenté par : Gabriel ORCUN

1. Contexte et objectif

Ce projet met en œuvre un data lake complet destiné à alimenter un système de RAG (Retrieval-Augmented Generation) sur des documents financiers. L'enjeu central est l'intégration de deux sources hétérogènes, des images de documents et des rapports annuels HTML qui n'ont ni le même format, ni le même mode d'acquisition, ni la même chaîne de traitement, mais qui doivent converger vers une représentation unique et interrogeable sémantiquement.

L'architecture suit le découpage canonique en trois zones raw → staging → curated. Chaque zone a une responsabilité unique : la zone raw préserve la donnée brute telle qu'ingérée, la zone staging valide sans transformer, la zone curated produit l'artefact métier. Ici du texte découpé en chunks et vectorisé. Cette séparation garantit qu'un bug de traitement en aval n'entraîne jamais la perte de la donnée source, et qu'un retraitement complet reste possible à tout moment.

Stack technique

Composant	Technologie	Rôle
Stockage objet	LocalStack (S3)	Simule S3 en local — 3 buckets
Vector store	Qdrant	Recherche sémantique (cœur du RAG)
Orchestration	Apache Airflow	2 DAGs — scheduling + incrémentalité
API	FastAPI	Exposition des 3 zones + recherche
OCR	EasyOCR	Image → texte (deep learning)
Parsing HTML	BeautifulSoup / lxml	10-K → texte propre
Embeddings	sentence-transformers	all-MiniLM-L6-v2 — 384 dimensions

2. Sources de données

Le sujet impose deux sources de nature différente. Ce choix n'est pas anodin : il force le pipeline à gérer à la fois une source statique (un dataset fichier) et une source dynamique (une API) ce qui a des conséquences directes sur la stratégie d'orchestration.

Sourc e	Type	Contenu	Chemin en zone raw
Datas et Kaggle	Fichier (image s)	~1600 images de documents financiers, 16 classes (invoice, budget, resume, memo, form, handwritten...)(script pour créer un dataset de 1600 images à partir du dataset suivant : https://www.kaggle.com/datasets/swatigupta555/financial-document-classification)	financial/{train,validation,test}/images/ *.jpeg
SEC EDGA R	API REST	Filings 10-K (rapports annuels) au format HTML, ~75 entreprises du S&P	sec_edgar/{TICKER}/{date}_10K.html

Un échantillon de 80 images (5 par classe) est versionné dans prof/data_demo/ afin que le projet soit testable sans téléchargement préalable du dataset Kaggle complet.

3. Architecture générale

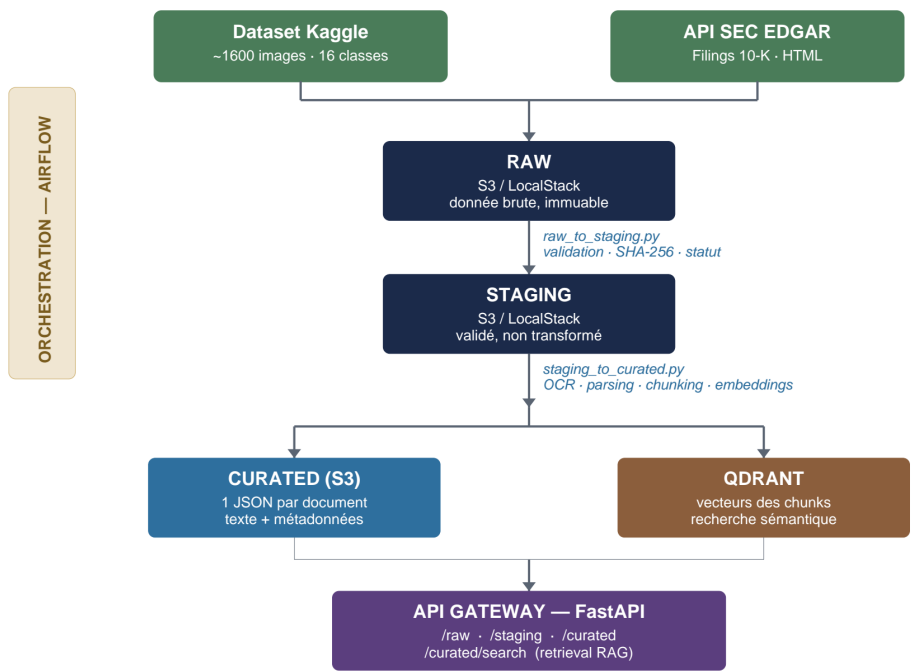


Figure 1 — Flux de bout en bout : deux sources hétérogènes, trois zones, double écriture en curated (S3 + Qdrant), exposition unifiée par l'API.

4. Les trois zones du data lake

4.1 Zone raw stockage brut

Le stockage se fait sur S3, simulé en local par LocalStack. Les deux sources y atterrissent sans aucune transformation. C'est le principe fondamental de la zone raw : elle est immuable et fait autorité. Si l'OCR change de moteur dans six mois, on rejoue le pipeline depuis raw sans avoir à re-télécharger quoi que ce soit.

Le script d'ingestion Kaggle effectue toutefois un split stratifié train/validation/test (80/10/10) *par classe*, afin de préserver l'équilibre des 16 classes dans chaque partition prérequis si un classifieur devait être entraîné ultérieurement sur ces données.

4.2 Zone staging validation et traçabilité

La zone staging valide sans transformer. La nature du contenu reste inchangée (une image reste une image, un HTML reste un HTML) : seuls des métadonnées et un verdict de validité sont ajoutés.

- Images : ouverture avec Pillow, extraction des dimensions et du format, détection des fichiers corrompus (une exception à l'ouverture entraîne le statut `invalid`).
- HTML SEC : vérification de non-vacuité et de la présence d'une structure HTML valide.

Chaque document reçoit un hash SHA-256 servant d'identifiant stable et un statut qui pilote la suite du pipeline :

Statut	Signification	Traitement en curated
<code>ready_for_ocr</code>	Image valide, lisible par Pillow	OCR EasyOCR
<code>ready_for_extraction</code>	HTML non vide et bien formé	Parsing BeautifulSoup
<code>invalid</code>	Fichier corrompu ou vide	Écarté — non copié en staging

Point de conception important. Le fichier `metadata.json` de chaque source est fusionné et non écrasé à chaque exécution. C'est ce qui rend les runs incrémentaux répétés possibles. Un nouveau passage du DAG SEC ajoute ses documents au catalogue existant au lieu de le remplacer, ce qui serait destructeur dans un contexte de scheduling quotidien.

5. Zone curated — convergence des deux sources

C'est le cœur technique du projet. Deux sources radicalement différentes doivent produire le même artefact final. Les chaînes d'extraction diffèrent, mais elles se rejoignent dès que du texte brut est disponible.

5.1 Extraction — deux chemins distincts

Source	Chaîne d'extraction	Justification du choix
SEC (HTML)	BeautifulSoup / lxml : suppression des balises script et style, extraction du texte, normalisation des espaces	Les 10-K sont du HTML lourd et bruité ; lxml est le parser le plus rapide sur des documents de cette taille
Financial (images)	EasyOCR — modèles de deep learning	Choisi plutôt que Tesseract car le dataset contient une classe handwritten : Tesseract est conçu pour du texte imprimé et échoue sur l'écriture manuscrite. EasyOCR la gère nettement mieux et ne dépend d'aucun binaire système externe

5.2 Chunking et vectorisation — le tronc commun

Une fois le texte obtenu, les deux flux suivent exactement le même traitement :

- Découpage en chunks (~1000 caractères, 150 de recouvrement). Le recouvrement est indispensable car sans lui, une information à cheval sur la frontière de deux chunks serait tronquée dans les deux, et donc irrécupérable par le retrieval. Un 10-K faisant plusieurs centaines de pages ne peut de toute façon pas être vectorisé d'un bloc.
- Embeddings avec sentence-transformers (all-MiniLM-L6-v2, 384 dimensions). Modèle exécuté localement et gratuitement : aucun appel à une API payante, le projet reste entièrement reproductible hors ligne.

5.3 Double écriture — S3 et Qdrant

La zone curated écrit dans deux destinations complémentaires, chacune répondant à un besoin distinct :

Destination	Contenu	Besoin couvert
S3 — bucket curated	Un JSON par document : texte complet + métadonnées	Exigence du sujet : accès simple et direct aux données ingérées (endpoint /curated)
Qdrant	Vecteurs des chunks, avec texte et métadonnées en payload	Recherche sémantique via /curated/search — la brique retrieval du RAG

Pourquoi Qdrant plutôt que ChromaDB ou FAISS ? Qdrant est un vrai service, déployé dans un conteneur Docker séparé au même titre que LocalStack. FAISS est une bibliothèque in-process et ChromaDB, dans son mode par défaut, reste couplé au processus Python qui l'alimente. Découpler le vector store du script qui l'écrit est à la fois plus proche d'une architecture de production et plus robuste. L'index survit à l'arrêt du pipeline, et l'API peut l'interroger sans partager de mémoire avec lui.

6. Orchestration — Apache Airflow

Airflow a été retenu plutôt que DVC. DVC est excellent pour le versionnement de données et la reproductibilité de pipelines ML, mais il ne fournit pas de scheduling natif. Or le sujet demande explicitement d'ingérer depuis une API à intervalle régulier et c'est précisément le domaine d'Airflow, qui apporte en plus les Variables et les XCom, deux mécanismes directement exploités ici pour l'incrémentalité.

6.1 Deux DAGs, deux natures de source

DAG	Schédule	Rôle et justification
sec_edgar_pipeline	@daily	Ingère les filings SEC de façon incrémentale . La source est une API vivante : de nouveaux filings apparaissent en continu, un scheduling périodique a donc du sens.
financial_bootstrap_pipeline	None (manuel)	Split, upload, staging et curation du dataset Kaggle. Ce dataset est statique : le planifier reviendrait à retraiter indéfiniment les mêmes 1600 images. Un déclenchement manuel unique suffit.

6.2 Mécanisme d'incrémentalité

Le DAG SEC illustre le point le plus intéressant de l'orchestration. Deux mécanismes Airflow y sont combinés pour éviter tout retraitement inutile :

- Une Airflow Variable (`sec_edgar_cursor`) mémorise la position atteinte dans la liste des entreprises d'un run à l'autre. Chaque déclenchement reprend là où le précédent s'est arrêté, au lieu de re-parcourir tout le catalogue SEC depuis le début.
- Les clés S3 effectivement ingérées lors du run sont transmises via XCom aux tâches `stage_new_filings` puis `curate_new_filings`. Ces tâches ne traitent que ces fichiers-là, et non l'intégralité du bucket. Sans ce mécanisme, chaque run quotidien re-parserait et re-vectoriserait tous les documents déjà présents avec un coût qui croîtrait linéairement à chaque exécution.

C'est cette combinaison curseur + XCom qui rend le DAG réellement idempotent et incrémental, et non simplement répété.

6.3 Configuration de déploiement

Airflow tourne en conteneur unique (`airflow standalone`, backend SQLite, `SequentialExecutor`). Ce choix est assumé car à l'échelle d'un projet académique, la stack complète Postgres + Redis + workers Celery apporterait une complexité de déploiement sans bénéfice mesurable, les DAGs étant de toute façon séquentiels.

7. API Gateway — FastAPI

L'API expose les trois zones du lake ainsi que la brique de recherche sémantique. Une documentation interactive est auto-générée sur <http://localhost:8000/docs>.

Endpoint	Description
GET /health	État des connexions S3 (raw / staging / curated) et Qdrant
GET /stats	Nombre d'objets et volumétrie par bucket ; nombre de vecteurs par collection Qdrant
GET /raw • GET /raw/{key}	Liste paginée des objets raw (prefix, limit, token) ; récupération d'un objet précis
GET /staging • GET /staging/{key}	Idem pour la zone staging
GET /curated • GET /curated/{key}	Liste paginée des documents curated ; récupération d'un document (texte + métadonnées)
GET /curated/search	Recherche sémantique — le retrieval du RAG. Embed la requête, interroge Qdrant, renvoie les passages les plus pertinents. Paramètres : q, top_k, et source optionnel (sec_edgar ou financial_documents) ; sans source, cherche dans les deux collections
POST /ingest	Niveau avancé. Chunk + embed + indexe chaque texte, un par un
POST /ingest_fast	Niveau avancé. Même contrat, mais embeddings et upsert Qdrant batchés, écritures S3 parallélisées

Exemple — recherche sémantique

```
# Question en langage naturel sur les filings SEC
curl "http://localhost:8000/curated/search\
?q=cybersecurity+risk+factors&source=sec_edgar&top_k=3"

# Question sur les documents financiers issus de l'OCR
curl "http://localhost:8000/curated/search\
?q=invoice+amount+due&source=financial_documents&top_k=5"

# Sans parametre 'source' : recherche dans les deux collections
curl "http://localhost:8000/curated/search?q=quarterly+revenue+growth&top_k=5"
```

Figure 2 — L'endpoint de recherche interroge indifféremment l'une ou l'autre source, ou les deux : c'est le bénéfice concret de la convergence opérée en zone curated.

8. Niveau avancé — optimisation de l'ingestion

Deux endpoints d'ingestion coexistent volontairement, afin de mesurer l'apport des optimisations plutôt que de le supposer.

8.1 Les trois optimisations de `/ingest_fast`

Opération	<code>/ingest</code>	<code>/ingest_fast</code>
Embeddings	Un appel <code>encode()</code> par document	Un seul appel <code>encode()</code> sur tous les chunks du batch — la vectorisation interne de <code>sentence-transformers</code> est bien plus efficace qu'un appel par chunk
Indexation Qdrant	Un <code>upsert()</code> par chunk	Un seul <code>upsert()</code> pour l'ensemble des points
Écritures S3	Séquentielles	Parallélisées sur 8 threads (<code>ThreadPoolExecutor</code>)

8.2 Résultats mesurés

Mesuré avec `scripts/benchmark_ingest.py` (médiane sur 3 répétitions) :

Taille du batch	<code>/ingest</code>	<code>/ingest_fast</code>	Gain
1 texte	0.030 s	0.032 s	-8 %
100 textes	2.30 s	0.30 s	+87 %

Lecture des résultats. Sur un seul élément, le batching n'apporte logiquement rien, il n'y a rien à batcher et le léger surcoût de -8 % correspond au coût d'initialisation du thread pool, qui n'est pas amorti. Ce résultat n'est pas un échec. Il confirme que l'optimisation agit bien là où on l'attend, et nulle part ailleurs.

Sur 100 éléments, le scénario réaliste d'un usage en production, le gain atteint +87 %, très au-delà du seuil de 30 % demandé par le sujet. L'essentiel de ce gain provient du batching des embeddings, `sentence-transformers` vectorise un lot de textes en une seule passe matricielle, là où 100 appels unitaires reconstruisent 100 fois le même surcoût fixe.

8.3 Parallélisation du pipeline `curated`

Le script `staging_to_curated.py` expose deux réglages de concurrence séparés, `--sec-workers` et `--financial-workers`. Ce n'est pas un caprice de configuration : les deux traitements ont des profils de charge opposés.

- Le traitement SEC est I/O-bound (appels S3, appels Qdrant) : les threads passent l'essentiel de leur temps à attendre le réseau. Monter à 6 threads concurrents donne environ 1,5× l'exécution séquentielle.
- L'OCR EasyOCR est CPU-bound et parallélise déjà en interne via PyTorch. Ajouter des threads Python par-dessus crée de la contention sur les mêmes cœurs : au-delà de 3 threads, le traitement ralentit au lieu d'accélérer.

Un réglage unique aurait donc nécessairement pénalisé l'un des deux flux.

9. Structure du projet et déploiement

9.1 Organisation du dépôt

```
api/
  main.py  config.py  clients.py  s3_utils.py
  routes/
    raw, staging, curated, health, stats

scripts/
  ingestion/
    sec_api.py      ingestion API SEC EDGAR
    upload.py       split + upload du dataset Kaggle
  raw_to_staging.py validation, hashing, statuts
  staging_to_curated.py OCR, parsing, chunking, embeddings, Qdrant
  s3_metadata.py    fusion incrementale des metadata.json
  benchmark_ingest.py mesure /ingest vs /ingest_fast

airflow/
  dags/
    sec_edgar_pipeline.py @daily, incremental
    financial_bootstrap_pipeline.py manuel
  Dockerfile  docker-compose.yml

docker/
  docker-compose.yml    LocalStack (S3) + Qdrant

prof/
  data_demo/            echantillon versionne : 80 images, 5 par classe

data/                  dataset Kaggle complet (non versionne)
```

9.2 Mise en route

Prérequis : Docker Desktop, **uv**, Python ≥ 3.10 .

```
# 1. Infrastructure : S3 (LocalStack) + Qdrant
cd docker && docker compose up -d && cd ..

# 2. Ingestion (demo rapide, quelques minutes)
uv run python scripts/ingestion/sec_api.py --limit 30
uv run python scripts/ingestion/upload.py \
  --source-dir prof/data_demo --output-dir data/financial_split

# 3. Pipeline raw -> staging -> curated
uv run python scripts/raw_to_staging.py
uv run python scripts/staging_to_curated.py

# 4. API
uv run uvicorn api.main:app --reload --port 8000

# 5. Airflow (optionnel) : UI sur http://localhost:8080
cd airflow && docker compose up -d --build
```

Figure 3 — Parcours de démonstration. Le pipeline complet (~2600 documents) demande environ 2 h sur CPU ; l'échantillon versionné (30 filings + 80 images) s'exécute en quelques minutes.

10. Limitations et pistes d'évolution

Limitation	Nature	Évolution envisagée
Génération absente	Le RAG s'arrête à la récupération : /curated/search renvoie les passages pertinents, mais aucun LLM n'est branché pour en synthétiser une réponse en langage naturel.	Ajouter un endpoint /ask qui injecte les top-k passages dans le prompt d'un LLM. La brique la plus coûteuse — le retrieval — est déjà en place.
Pas de classifieur	Les 16 labels du dataset Kaggle sont conservés en métadonnée, mais aucun modèle n'a été entraîné dessus (piste bonus ML/DL du sujet).	Le split stratifié train/val/test est déjà produit en zone raw : les données sont prêtes pour un entraînement de classification d'images.
Airflow mono-conteneur	SQLite + SequentialExecutor : pas de parallélisme entre tâches, pas de haute disponibilité.	Passage à Postgres + Celery workers si le volume ou le nombre de DAGs augmentait.

11. Conclusion

Le data lake est fonctionnel de bout en bout. Deux sources hétérogènes, une API et un dataset fichier, du HTML et des images, traversent un pipeline raw → staging → curated et convergent vers une représentation unique, vectorisée et interrogeable sémantiquement au travers d'une API unifiée.

Les décisions structurantes du projet ont toutes été prises en fonction d'une contrainte concrète, et non par défaut :

- EasyOCR plutôt que Tesseract, parce que le dataset contient une classe manuscrite que Tesseract ne sait pas traiter.
- Qdrant plutôt que FAISS ou ChromaDB, pour découpler le vector store du processus qui l'alimente.
- Airflow plutôt que DVC, parce que l'ingestion depuis une API vivante exige un scheduling natif, et que Variables et XCom permettent une incrémentalité réelle.
- Deux réglages de concurrence distincts, parce qu'un flux I/O-bound et un flux CPU-bound n'ont pas le même optimum.

Le gain de +87 % mesuré sur /ingest_fast à l'échelle de 100 documents valide la démarche d'optimisation, tout en montrant par le léger surcoût observé sur un élément unique, qu'une optimisation n'a de sens qu'au regard du régime d'usage pour lequel elle est conçue.

La principale extension naturelle reste le branchement d'un LLM en aval du retrieval, pour transformer ce moteur de recherche sémantique en un véritable système de question-réponse génératif.